

Redes y Programación de Sockets

Antonio Espín Herranz

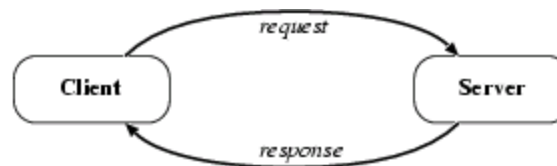
Contenidos

- Fundamentos de sockets
- Multiplexación de E/S
- Servidores concurrentes
- Protocolos y serialización

Fundamentos de sockets

Socket

- Un socket se podría equiparar a un enchufe de la red eléctrica.
- Los vamos a utilizar para comunicar procesos que pueden estar ejecutándose en la misma máquina localhost o en máquinas distintas.
- Necesitamos dos parámetros para conectar la dirección IP y el número de puerto.
- Tener en cuenta que los primeros 1024 número de puerto están reservados. Por ejemplo:
25 → smtp, 80 → http, 20, 21 → FTP.
- Los Sockets nos permiten trabajar con una arquitectura Cliente / Servidor.



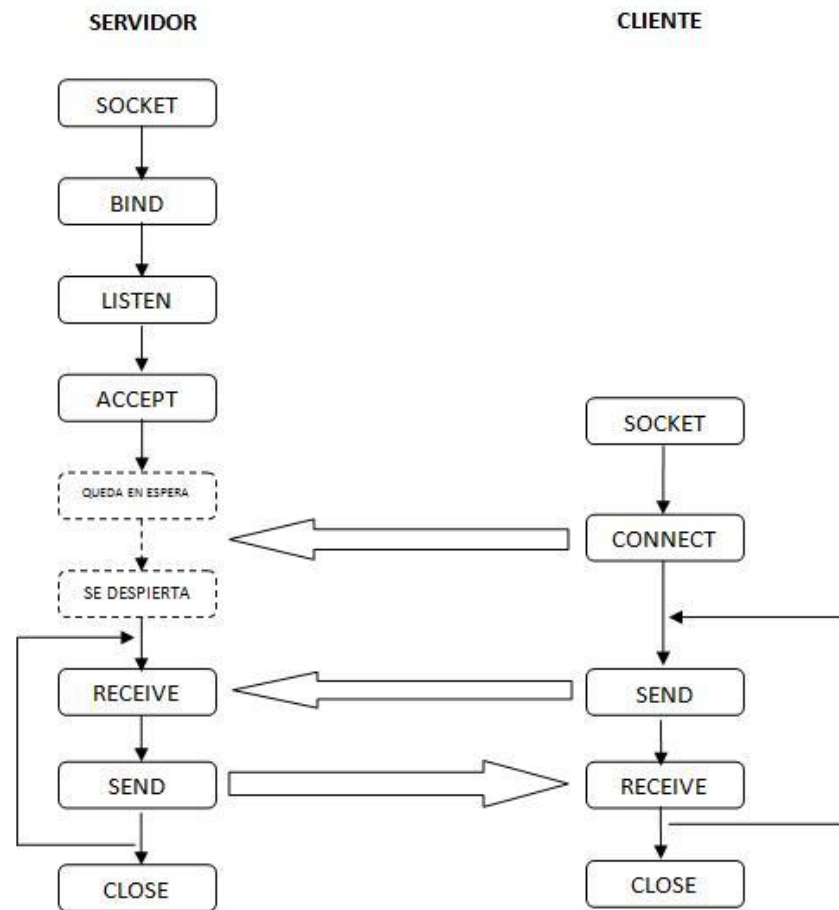
Tipos de Sockets

- Las dos clases de Sockets más importantes:
 - **Socket de flujo:**
 - Flujo de datos de dos vías, confiable, y sin duplicados sin límites de grabación. El flujo opera en forma parecida a una conversación telefónica. El tipo del socket es ***SOCK_STREAM***, el cual en el dominio de Internet usa **TCP** (Transmission Control Protocol).
 - Sobre este tipo se basan varios servicios de Internet:
 - La Web, Correo Electrónico, Telnet, FTP. (NO Permiten pérdidas)
 - **Socket de datagrama:**
 - soporta un flujo de mensajes de dos vías. En un socket de datagrama podría recibir mensajes en diferente orden de la secuencia de la cual los mensajes fueron enviados. No garantiza que los datos sean entregados. El tipo de socket es ***SOCK_DGRAM***, el cual en el dominio de internet usa **UDP** (User Datagram Protocol).
 - Sobre este tipo se basan varios servicios de Internet:
 - DNS, Voz IP, Video Conferencia. (SI, Permiten pérdidas)

Tipos de Dominios

- El dominio de comunicación nos dice como se va a realizar la comunicación de los procesos que se van a intercomunicar, o mejor dicho, en que ambiente.
- Si los procesos se comunicarán bajo la forma de un único sistema (tipo root de Unix), el dominio de comunicación será **AF_UNIX**, si los procesos se comunicarán como sistemas independientes y estos se hallan unidos mediante una red TCP/IP, el dominio de comunicación será **AF_INET**.
- En nuestro caso vamos a utilizar el dominio (también llamado familia de protocolos) **AF_INET**.

Cliente / Servidor



Cliente / Servidor

- Pasos que realiza el Servidor:

- Crear el **Socket**.
- Configurar los datos de la dirección del Servidor (IP, puerto, etc).
- Ligar el socket con la dirección. (**Bind**).
- Poner el Socket del Servidor a la escucha para recibir peticiones por parte del Cliente. (**Listen**).
- Aceptar peticiones de los clientes (**Accept**).
- Una vez que hemos conectado con el cliente creamos un proceso hijo con **fork** y le podemos enviar datos mediante **Send**

- Pasos que realiza el Cliente

- Puede recuperar la información del host con el que quiere conectar a partir de la información de la función **gethostbyname()**.
 - **gethostbyname("localhost")**
- Crear un **Socket** para comunicarse.
- Crear la dirección destino.
- Realizar **connect** para conectar con el Servidor.
- Recibir datos del Servidor mediante **recv**.

Cliente / Servidor

- La puesta en marcha se realiza de la siguiente manera:
 - Lanzar el Servidor.
 - Después lanzar el cliente indicando en este caso localhost (se manda como parámetro de main()).
 - También se puede probar un cliente, mediante el comando Telnet. (desde una consola de MSDOS)
 - Telnet localhost 3490 (Tenemos que indicar el puerto en el que hayamos arrancado el Servidor)

Estructuras

// Esta estructura representa un Socket:

```
struct sockaddr_in {  
    /* Familia de la Dirección */  
    short int sin_family;  
    /* Puerto */  
    unsigned short int sin_port;  
    /* Dirección de Internet */  
    struct in_addr sin_addr;  
    unsigned char sin_zero[8];  
};
```

Estructuras

// Esta estructura representa una dirección IP.

```
struct in_addr {  
    unsigned long s_addr;  
};
```

Estructuras

// Esta estructura representa un HOST.

```
struct hostent {  
    char *h_name; /* El nombre oficial del nodo. */  
    char **h_aliases; /* Lista de Alias. */  
    int h_addrtype; /* Tipo de dirección del nodo. */  
    int h_length; /* Longitud de la dirección. */  
    char **h_addr_list; /* Lista de direcciones del nombre del servidor. */  
    #define h_addr h_addr_list[0] /* Dirección, para la compatibilidad con anteriores. */  
};
```

- La podemos utilizar para conocer los datos de un host a partir de su nombre:

```
struct hostent *he;  
he = gethostbyname("localhost");
```

socket()

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int socket(int domain,int type,int protocol);
```

- **domain.** Se podrá establecer como AF_INET (para usar los protocolos ARPA de Internet), o como AF_UNIX (si se desea crear sockets para la comunicación interna del sistema).
- **type.** Aquí se debe especificar la clase de socket que queremos usar (de Flujos o de Datagramas). Las variables que deben aparecer son SOCK_STREAM o SOCK_DGRAM según queramos usar sockets de Flujo o de Datagramas, respectivamente.
- **protocol.** Aquí, simplemente se puede establecer el protocolo a 0.

bind()

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int bind(int fd, struct sockaddr *my_addr,int addrlen);
```

- **fd.** Es el descriptor de fichero socket devuelto por la llamada a `socket()`.
- **my_addr.** es un puntero a una estructura `sockaddr`
- **addrlen.** contiene la longitud de la estructura `sockaddr` a la cuál apunta el puntero `my_addr`. Se debera establecer como `sizeof(struct sockaddr)`.
- La llamada `bind()`: Su función esencial es asociar un socket con un puerto (de nuestra máquina). Análogamente `socket()`, devolverá -1 en caso de error.

connect()

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int connect(int fd, struct sockaddr *serv_addr, int addrlen);
```

- **fd.** Deberá configurarse como el fichero descriptor del socket, el cuál fue devuelto por la llamada a `socket()`.
- **serv_addr.** Es un puntero a la estructura `sockaddr` la cuál contiene la dirección IP destino y el puerto.
- **addrlen.** Análogamente de lo que pasaba con `bind()`, este argumento debera establecerse como `sizeof(struct sockaddr)`.
- La función `connect()` se usa para conectarse a un puerto definido en una dirección IP, la utiliza el cliente para conectar con el Servidor. Devolverá -1 si ocurre algún error.

listen()

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int listen(int fd,int backlog);
```

- **fd.** Es el fichero descriptor del socket, el cual fue devuelto por la llamada a socket().
- **backlog.** Es el número de conexiones permitidas.
- La función listen() se usa si se están esperando conexiones entrantes, lo cual significa, si se quiere, que alguien pueda conectarse a nuestra máquina. La ejecutará el Servidor.
- Después de listen() se ejecutará accept() para aceptar a un cliente.

accept()

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int accept(int fd, void *addr, int *addrlen);
```

- **fd.** Es el fichero descriptor del socket, que fue devuelto por la llamada a `listen()`.
- **addr.** Es un puntero a una estructura `sockaddr_in` en la que se pueda determinar que nodo nos está contactando y desde que puerto.
- **addrlen.** Es la longitud de la estructura a la que apunta el argumento `addr`, por lo que conviene establecerlo como `sizeof(struct sockaddr_in)`, antes de que su dirección sea pasada a `accept()`.

send()

```
#include <sys/types.h>  
#include <sys/socket.h>
```

```
int send(int fd,const void *msg,int len,int flags);
```

- **fd.** Es el fichero descriptor del socket, con el cual se desea enviar datos.
- **msg.** Es un puntero apuntando al dato que se quiere enviar.
- **len.** es la longitud del dato que se quiere enviar (en bytes).
- **flags.** deberá ser establecido a 0.

recv()

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int recv(int fd, void *buf, int len, unsigned int flags);
```

- **fd.** Es el descriptor del socket por el cual se leerán datos.
- **buf.** Es el búfer en el cual se guardará la información a recibir.
- **len.** Es la longitud máxima que podrá tener el búffer.
- **flags.** Por ahora, se deberá establecer como 0.

close()

```
#include <unistd.h>  
close(fd);
```

- La función close() es usada para cerrar la conexión de nuestro descriptor de socket.
- Si llamamos a close() no se podrá escribir o leer usando ese socket, y si alguien trata de hacerlo recibirá un mensaje de error.

Servidores Concurrentes

Modelo de hilos

Modelo de procesos

fork()

Ejecución de comandos

- Desde un programa C, podemos ejecutar comandos del sistema operativo.
- `#include <unistd.h>`
- `int system(const char *cmdstring);`
 - **system()** crea un proceso que ejecuta un shell y le pasa el comando para que lo ejecute.
- Devuelve el código retornado por el comando de shell, 127 si no pudo ejecutar el shell y -1 en caso de otro error.

Creación de procesos

- La llamada al sistema **fork()** crea un proceso:
- `#include <unistd.h>`
- `pid_t fork(void);`
- Se crea un proceso **idéntico al padre**.
- `fork()` devuelve 0 al proceso hijo, el PID del proceso hijo al padre, -1 en caso de error.
- Cada proceso queda identificado en la máquina por un número, llamado pid (process id).

Funciones de terminación

- **_exit()** vuelve al kernel inmediatamente. Definida por POSIX:
 - #include <unistd.h>
 - void _exit(int status);
- **exit()** realiza antes cierto “limpiado” (e.g. terminar de escribir los buffers a disco). Es ANSI C:
 - #include <stdlib.h>
 - void exit(int status);
- **abort()** *el proceso se envía a sí mismo la señal SIGABRT. Termina y produce un **core dump** (volcado de memoria).*

Espera por el proceso hijo

- `#include <sys/types.h>`
- `#include <sys/wait.h>`
- `pid_t wait(int *statloc);`
- `pid_t waitpid(pid_t pid, int *statloc, int options);`
- Suspende al proceso que la ejecuta hasta que alguno de sus hijos termina.
- Si algún hijo ha terminado se devuelve el resultado inmediatamente.
- El valor retornado por el proceso hijo puede deducirse del valor devuelto en `statloc`.

Ejemplo

```
if ( fork()==0 ){  
    printf ("Proceso hijo!!! \n");  
    exit(1);  
} else { // Proceso padre espera a que termine el hijo  
    int mark;  
    wait(&mark);  
}
```

Ejecutar otro programa

- **exec()** se emplea para lanzar un programa de un fichero.
- El programa en ejecución en el proceso llamante es sustituido por el programa que se ejecuta, el PID no cambia.
- `char* argumentos [] = { "ls", "-l", "/usr/include", 0 };`
- `execv ( "/bin/ls", argumentos );`
- Las llamadas retornan un valor no nulo si no se puede ejecutar el programa.

Modelo Reactivo

select / epoll